

Table of contents

Complex Disease Models	1
Introduction	1
Setup	2
SEIR model	2
Comparing SIR and SEIR	5
SIS model	6
Custom multi-stage infection	10
Edge-level dynamics	13
Simulation validation	15
Summary	19

Complex Disease Models

Simon Frost 2026-05-13

- [Introduction](#)
- [Setup](#)
- [SEIR model](#)
- [Comparing SIR and SEIR](#)
- [SIS model](#)
- [Custom multi-stage infection](#)
- [Edge-level dynamics](#)
- [Simulation validation](#)
- [Summary](#)

Introduction

The expanded edge-based formulation is not limited to SIR — it handles arbitrary disease progression graphs. Each disease stage that can transmit infection gets its own edge-level probability φ , and the package automatically constructs the full system of ODEs.

This vignette demonstrates:

- SEIR: adding a latent (exposed) period
- SIS: endemic dynamics without lasting immunity
- Multi-stage infections: acute and chronic infectious periods with different transmission rates

Setup

```
using EdgeBasedModels
using ModelingToolkit
using OrdinaryDiffEq
using Symbolics
using Plots
```

```
@parameters β γ κ
pgf = poisson_pgf(κ)
κ_val = 5.0
ψ(x) = exp(κ_val * (x - 1))
tspan = (0.0, 40.0)
γ_val = 0.25
R0_target = 2.0
seed_fraction = 0.01
T_val = R0_target / κ_val
β_val = T_val * γ_val / (1 - T_val)
```

0.16666666666666669

SEIR model

[!NOTE]

$R_0 = 2$ anchor. Generic SIR/SIS comparisons use $\gamma = 0.25$, 1% seeding, and Poisson $\kappa = 5$, giving $T = 2/5$ and per-edge $\beta = 1/6$. The SEIR latent period changes timing but not this transmissibility-based R_0 .

The SEIR model adds an exposed (latent) class E between S and I . Exposed individuals are infected but not yet infectious. The transition rate from E to I is σ (so the mean latent period is $1/\sigma$).

$$S \xrightarrow{\text{infection}} E \xrightarrow{\sigma} I \xrightarrow{\gamma} R$$

```
@parameters σ
model_seir = build_seir(pgf, σ, β, γ)
eqs = ModelingToolkit.equations(model_seir.system)
println("SEIR: $(length(eqs)) ODEs")
for eq in eqs
    println(" ", eq)
end
```

SEIR: 7 ODEs

```
Differential(t, 1)(pop_R(t)) ~ pop_I(t)*γ
Differential(t, 1)(pop_I(t)) ~ -pop_I(t)*γ + pop_E(t)*σ
Differential(t, 1)(pop_E(t)) ~ -pop_E(t)*σ + exp((-1 + θ(t))*κ)*phi_I(t)*β*κ*(1 -
ρ)
Differential(t, 1)(phi_R(t)) ~ phi_I(t)*γ
Differential(t, 1)(phi_I(t)) ~ phi_E(t)*σ - phi_I(t)*(β + γ)
Differential(t, 1)(phi_E(t)) ~ -phi_E(t)*σ + phi_S(t)*phi_I(t)*β*κ
Differential(t, 1)(θ(t)) ~ -phi_I(t)*β
```

The SEIR model has 5 ODEs: θ , φ_E , φ_I , φ_R , and R .

[!NOTE]

At the population level, the model tracks $S = \psi(\theta)$, R , and the combined “infected” fraction $E + I = 1 - S - R$. The edge-level variables φ_E and φ_I track the probability that a random neighbor is in each stage, providing more granular information than the population-level quantities.

```
σ_val = 0.2 # mean latent period = 5 days
ic_seir = default_initial_conditions(model_seir; seed_fraction = seed_fraction)
prob_seir = ODEProblem(
    model_seir.system,
    merge(ic_seir, Dict{β ⇒ β_val, γ ⇒ γ_val, σ ⇒ σ_val, κ ⇒ κ_val}),
    tspan,
)
sol_seir = solve(prob_seir, Tsit5(); saveat = 0.5)
```

retcode: Success

Interpolation: 1st order linear

t: 81-element Vector{Float64}:

0.0

0.5

1.0

1.5

2.0

2.5

3.0

3.5

4.0

4.5

?

36.0
36.5
37.0
37.5
38.0
38.5
39.0
39.5
40.0

u: 81-element Vector{Vector{Float64}}:

```
[0.0, 0.0, 0.01, 0.0, 0.0, 0.010000000000000009, 1.0]
[5.81984086357102e-5, 0.0008996512591088815, 0.009229032824458099, 5.6636401526874715e-
5, 0.0008634556651998017, 0.009229032824458108, 0.999962242398982]
[0.0002182887755140383, 0.001639694725541273, 0.008824849244448799, 0.00020699040376162206
[0.0004634856611399941, 0.0022680328648803983, 0.008682641980065214, 0.000428839849739497,
[0.0007820756480752912, 0.0028187266411495377, 0.008729919172285125, 0.0007071119337892074
[0.0011659556624731386, 0.00331618276894121, 0.008916641993600337, 0.0010317054622516737, 0
[0.0016096410736669972, 0.0037779862034190053, 0.009208551328068974, 0.00139602452993727, 0
[0.00210950734008383, 0.00421705813700124, 0.009582087781489198, 0.001795865499489187, 0.00
[0.002663356703682486, 0.004642879089875887, 0.010021526064601471, 0.002228792663107068, 0.
[0.003269977415324207, 0.005062731636598092, 0.01051606581513131, 0.002693502420348398, 0.0
[
[0.2752312148878979, 0.07811065608392868, 0.10761823252342488, 0.18237507321205312, 0.04938
[0.2850558930688854, 0.07906070067341138, 0.10787305414237072, 0.18857706467971827, 0.04982
[0.294992492233122, 0.0799176911947884, 0.10796811421498691, 0.19482813834577262, 0.0501966
[0.30502927782899053, 0.08067755145820124, 0.10790512291583026, 0.20112041564687094, 0.0505
[0.3151543523836173, 0.08133599666203688, 0.10768784873031315, 0.20744629283859378, 0.05074
[0.3253556555028682, 0.08188853339292768, 0.10732211845470369, 0.21379844099544507, 0.05091
[0.3356204312699595, 0.08233195001272543, 0.10681325507124194, 0.22016882688250647, 0.05100
[0.3459340558996973, 0.08266815809864733, 0.10615957788360707, 0.22654638008144284, 0.05102
[0.35628297976933054, 0.08289663224359023, 0.10536537944971427, 0.2329221587705292, 0.05097
```

```
S_seir = compartment(sol_seir, model_seir, :S)
R_seir = compartment(sol_seir, model_seir, :R)
EI_seir = 1.0 .- S_seir .- R_seir # E + I combined
```

81-element Vector{Float64}:

```
0.010000000000000009
0.010128684075262608
0.010464543963436615
0.010950674854890619
0.011548645852730206
```

```

0.01223282477930886
0.012986537617558538
0.013799145919390274
0.014664405276611024
0.015578797468860919
0.1857297269416157
0.18693436850259887
0.18788602910319113
0.1885825278332674
0.18902349441443472
0.1892102780019254
0.18914483724465186
0.18882736708755465
0.1882616431398083

```

Comparing SIR and SEIR

```

# Build SIR for comparison
model_sir = build_sir(pgf, β, γ; form = :expanded)
ic_sir = default_initial_conditions(model_sir; seed_fraction = seed_fraction)
prob_sir = ODEProblem(
    model_sir.system,
    merge(ic_sir, Dict{β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val}),
    tspan,
)
sol_sir = solve(prob_sir, Tsit5(); saveat = 0.5)

S_sir = compartment(sol_sir, model_sir, :S)
I_sir = compartment(sol_sir, model_sir, :I)
R_sir = compartment(sol_sir, model_sir, :R)

```

```

81-element Vector{Float64}:
 0.0
 0.0014399021800027027
 0.003304790164502933
 0.005672864702126067
 0.008637402072483787
 0.01230795211693954
 0.016811057190712315

```

```

0.022289842075696033
0.02890284518272256
0.03681949639159166
0.7960656312947881
0.7964958825820978
0.7968821762595987
0.7972287945897161
0.7975396152211061
0.7978182242985103
0.7980679164627565
0.7982916948507573
0.7984922710955114

```

```

plot(sol_sir.t, I_sir, label = "SIR (E+I)", linewidth = 2, color = :red)
plot!(sol_seir.t, EI_seir, label = "SEIR (E+I)", linewidth = 2, color = :orange,
      linestyle = :dash)
plot!(sol_sir.t, R_sir, label = "R (SIR)", linewidth = 1, color = :green)
plot!(sol_seir.t, R_seir, label = "R (SEIR)", linewidth = 1, color = :green,
      linestyle = :dash)
xlabel!("Time")
ylabel!("Fraction of population")
title!("SIR vs SEIR ( $\sigma$ =$ $\sigma_{val}$ ,  $\beta$ =$ $\beta_{val}$ ,  $\gamma$ =$ $\gamma_{val}$ )")

```

The exposed period delays the epidemic peak but does not change the final size (since R_0 depends only on the infectious period and transmission rate, not the latent period).

SIS model

In the SIS model, individuals recover to the susceptible state — there is no lasting immunity. This leads to an endemic equilibrium where $I > 0$ persists indefinitely.

```

model_sis = build_sis(pgf,  $\beta$ ,  $\gamma$ )
eqs_sis = ModelingToolkit.equations(model_sis.system)
println("SIS: $(length(eqs_sis)) ODE")
for eq in eqs_sis
    println("  ", eq)
end

```

SIS: 1 ODE

Differential(t, 1)($\theta(t)$) ~ (($\theta(t)$)*exp(0) - exp((-1 + $\theta(t)$))* κ)*(1 - ρ))* β - (1 - $\theta(t)$)*exp(0)* γ) / (-exp(0))

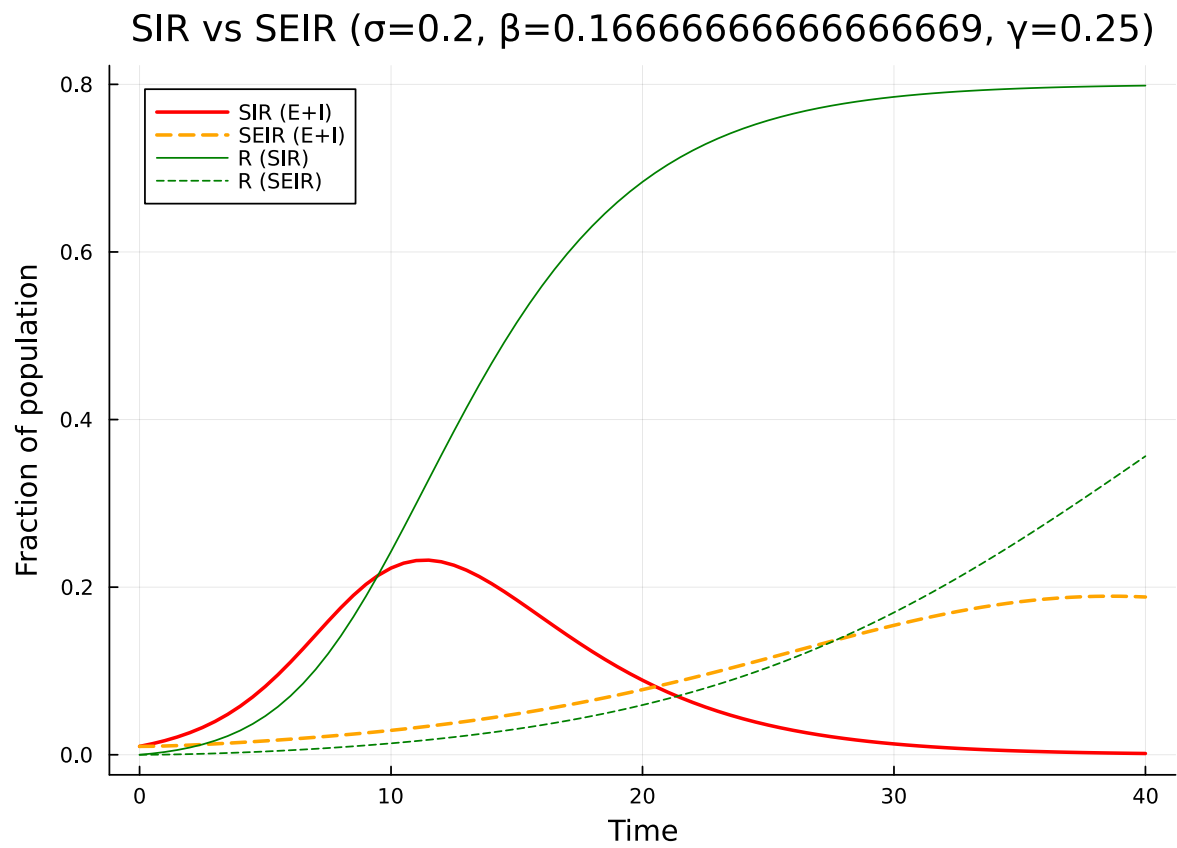


Figure 1: Figure 1: SIR vs SEIR: the exposed period delays and flattens the epidemic peak.

The SIS model has just 1 ODE for θ , with S and I as observables.

```
prob_sis = ODEProblem(  
    model_sis.system,  
    merge(  
        default_initial_conditions(model_sis; seed_fraction = seed_fraction),  
        Dict( $\beta \Rightarrow \beta\_val$ ,  $\gamma \Rightarrow \gamma\_val$ ,  $\kappa \Rightarrow \kappa\_val$ ),  
    ),  
    (0.0, 80.0),  
)  
sol_sis = solve(prob_sis, Tsit5(); saveat = 0.5)  
  
S_sis = compartment(sol_sis, model_sis, :S)  
I_sis = compartment(sol_sis, model_sis, :I)
```

161-element Vector{Float64}:

```
0.0100000000000000009  
0.014564261464581452  
0.020122497266740424  
0.02686819215835068  
0.03501973204376374  
0.04481820577203155  
0.056532640439969084  
0.07043490689530896  
0.08678843645178125  
0.10582692954207096  
0.8002032808029537  
0.8002022275436793  
0.8002012121953894  
0.8002002831237899  
0.8001994935913623  
0.8001989017571696  
0.8001985706764562  
0.8001985682999876  
0.8001989674730821
```

```
plot(sol_sis.t, S_sis, label = "S", linewidth = 2, color = :blue)  
plot!(sol_sis.t, I_sis, label = "I", linewidth = 2, color = :red)  
xlabel!("Time")  
ylabel!("Fraction of population")  
title!("SIS on Poisson network ( $\beta=\beta\_val$ ,  $\gamma=\gamma\_val$ ,  $\kappa=\kappa\_val$ )")
```

SIS on Poisson network ($\beta=0.16666666666666669$, $\gamma=0.25$, $\kappa=$

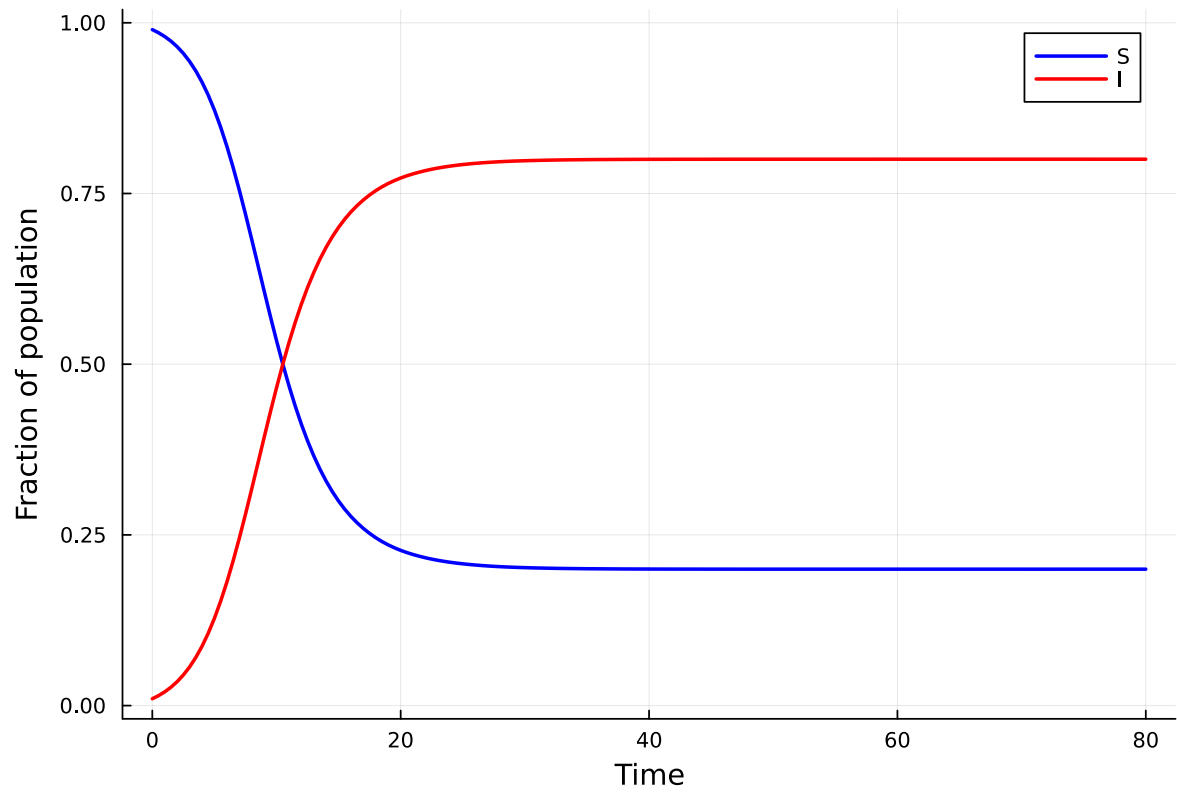


Figure 2: Figure 2: SIS model showing endemic equilibrium.

Unlike SIR, the infection reaches an endemic equilibrium where a constant fraction of the population remains infected.

Custom multi-stage infection

For diseases with multiple infectious stages (e.g., acute followed by chronic), we build a Disease-Progression directly. Each stage can have a different transmission rate.

Consider a model where infection proceeds: $I_1 \xrightarrow{\gamma_1} I_2 \xrightarrow{\gamma_2} R$, with I_1 (acute) having transmission rate $\beta_1 = 0.8$ and I_2 (chronic) having $\beta_2 = 0.15$.

```
@parameters  $\beta_1$   $\beta_2$   $\gamma_1$   $\gamma_2$ 

progression = DiseaseProgression(
  [
    DiseaseStage(:I1; transmission_rate =  $\beta_1$ ),
    DiseaseStage(:I2; transmission_rate =  $\beta_2$ ),
    DiseaseStage(:R; transmission_rate = 0),
  ],
  [
    DiseaseTransition(:I1, :I2,  $\gamma_1$ ),
    DiseaseTransition(:I2, :R,  $\gamma_2$ ),
  ];
  entry = :I1,
)

model_multi = build_edge_system(
  StaticConfigurationModel(pgf, progression);
  form = :expanded,
)

eqs_multi = ModelingToolkit.equations(model_multi.system)
println("Two-stage infection: $(length(eqs_multi)) ODEs")
for eq in eqs_multi
  println(" ", eq)
end
```

Two-stage infection: 7 ODEs

```
Differential(t, 1)(pop_R(t)) ~ pop_I2(t)* $\gamma_2$ 
Differential(t, 1)(pop_I2(t)) ~ pop_I1(t)* $\gamma_1$  - pop_I2(t)* $\gamma_2$ 
Differential(t, 1)(pop_I1(t)) ~ -pop_I1(t)* $\gamma_1$  + (phi_I1(t)* $\beta_1$  + phi_I2(t)* $\beta_2$ )*exp((-
1 +  $\theta(t)$ )* $\kappa$ )* $\kappa$ *(1 -  $\rho$ )
Differential(t, 1)(phi_R(t)) ~ phi_I2(t)* $\gamma_2$ 
```

$$\begin{aligned} \text{Differential}(t, 1)(\text{phi_I2}(t)) &\sim \text{phi_I1}(t) * \gamma_1 - \text{phi_I2}(t) * (\beta_2 + \gamma_2) \\ \text{Differential}(t, 1)(\text{phi_I1}(t)) &\sim -\text{phi_I1}(t) * \beta_1 - \text{phi_I1}(t) * \gamma_1 + (\text{phi_I1}(t) * \beta_1 + \text{phi_I2}(t) * \beta_2) \\ \text{Differential}(t, 1)(\theta(t)) &\sim -\text{phi_I1}(t) * \beta_1 - \text{phi_I2}(t) * \beta_2 \end{aligned}$$

```
ic_multi = default_initial_conditions(model_multi)
prob_multi = ODEProblem(
    model_multi.system,
    merge(ic_multi, Dict{β₁ ⇒ 0.8, β₂ ⇒ 0.15, γ₁ ⇒ 0.5, γ₂ ⇒ 0.05, κ ⇒ κ_val}),
    (0.0, 150.0),
)
sol_multi = solve(prob_multi, Tsit5(); saveat = 0.5)
```

retcode: Success

Interpolation: 1st order linear

t: 301-element Vector{Float64}:

```
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
⋮
146.0
146.5
147.0
147.5
148.0
148.5
149.0
149.5
150.0
```

u: 301-element Vector{Vector{Float64}}:

```
[0.0, 0.0, 0.001, 0.0, 0.0, 0.0010000000000000009, 1.0]
[5.738048418178308e-6, 0.0006074289067175093, 0.004733973840844089, 5.055808595410123e-
6, 0.0005154203843888398, 0.003954467387176528, 0.999127802784181]
[4.5835058336184574e-5, 0.003185453329591621, 0.01944833932768127, 3.7474245116546364e-
5, 0.002522362518564858, 0.015731753018128364, 0.9956119620662007]
[0.000224416520689902, 0.013259166960191637, 0.0733324025309947, 0.00017613199019476903, 0.
```

```
[0.0009031707292402983, 0.047119145838594625, 0.22242895198094706, 0.0006899272561738217, 0.0029782491203179137, 0.1274379239796132, 0.4315152154314008, 0.002197488639282697, 0.0907601579914419534, 0.24587525195305462, 0.5265440350163628, 0.005309001970863346, 0.157296837221577967, 0.36801177168754295, 0.4989659119019571, 0.009946689205931513, 0.20859676586458526, 0.473555599457935, 0.42707224258101933, 0.015541075593219563, 0.2348795257276579884, 0.5577611702201123, 0.350873140208969, 0.021528908396735817, 0.241607, -5.598019209861933e-7, 0.1065290249380783]
[0.9876738885352343, 0.0008468190786031991, 3.086165449072228e-8, 0.09505005870732501, 2.647, -1.3139193068962344e-7, 0.1065292732002857]
[0.9876947964949998, 0.0008259205044002825, 7.244536215231447e-9, 0.09505006675152172, 6.208, -1.3139193068962344e-7, 0.1065292732002857]
[0.9877151880319963, 0.0008055430867335449, -2.8284168710203266e-8, 0.09505007885308266, -2.424387725940906e-7, 5.131033097308281e-7, 0.10652964668305862]
[0.9877350766291073, 0.000785655552763223, -3.0957577525862795e-8, 0.09505007976394875, -2.6535873257737233e-7, 5.616119301592154e-7, 0.10652967479285332]
[0.9877544746957446, 0.0007662460153893802, -2.0910104794117094e-9, 0.09505006993190133, -1.7943231613852584e-8, 3.798075121386589e-8, 0.1065293713492974]
[0.9877733936424824, 0.0007473201691063291, 1.5272067452685957e-8, 0.09505006401786573, 1.37, -2.769880987487556e-7, 0.10652918882532261]
[0.9877918450906276, 0.0007288712835294041, 8.82450784774876e-9, 0.09505006621383644, 7.5628, -1.600397547487854e-7, 0.10652925659638608]
[0.9878098407147984, 0.0007108843556161476, -1.3057780320251232e-8, 0.09505007366701893, -1.1193010796325787e-7, 2.3689391747692176e-7, 0.10652948661819425]
[0.9878273922429246, 0.0006933361096663086, -2.1315830680702043e-8, 0.0950500764798295, -1.8271068798330962e-7, 3.8669432147499287e-7, 0.10652957342670281]
```

```
S_multi = compartment(sol_multi, model_multi, :S)
I_multi = compartment(sol_multi, model_multi, :I)
R_multi = compartment(sol_multi, model_multi, :R)
```

301-element Vector{Float64}:

```
0.0
5.738048418178308e-6
4.5835058336184574e-5
0.000224416520689902
0.0009031707292402983
0.0029782491203179137
0.007601579914419534
0.015296837221577967
0.025859676586458526
0.038795257276579884
```

⌵

```

0.9876738885352343
0.9876947964949998
0.9877151880319963
0.9877350766291073
0.9877544746957446
0.9877733936424824
0.9877918450906276
0.9878098407147984
0.9878273922429246

```

```

plot(sol_multi.t, S_multi, label = "S", linewidth = 2, color = :blue)
plot!(sol_multi.t, I_multi, label = "I1 + I2", linewidth = 2, color = :red)
plot!(sol_multi.t, R_multi, label = "R", linewidth = 2, color = :green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("Two-stage Infection ( $\beta_1=0.8$ ,  $\beta_2=0.15$ )")

```

Edge-level dynamics

One of the unique features of edge-based models is access to the edge-level variables. These show the probability that a random neighbor is in each disease stage.

```

# Extract  $\varphi$  variables from the multi-stage solution
 $\varphi$ _vars = Dict{String,Any}()
for (name, var) in model_multi.variables
    if startswith(String(name), " $\varphi$ ")
         $\varphi$ _vars[String(name)] = var
    end
end
println("Edge-level variables:")
for (name, _) in sort(collect( $\varphi$ _vars); by = first)
    println("  $name")
end

```

Edge-level variables:

```

 $\varphi$ _I1
 $\varphi$ _I2
 $\varphi$ _R

```

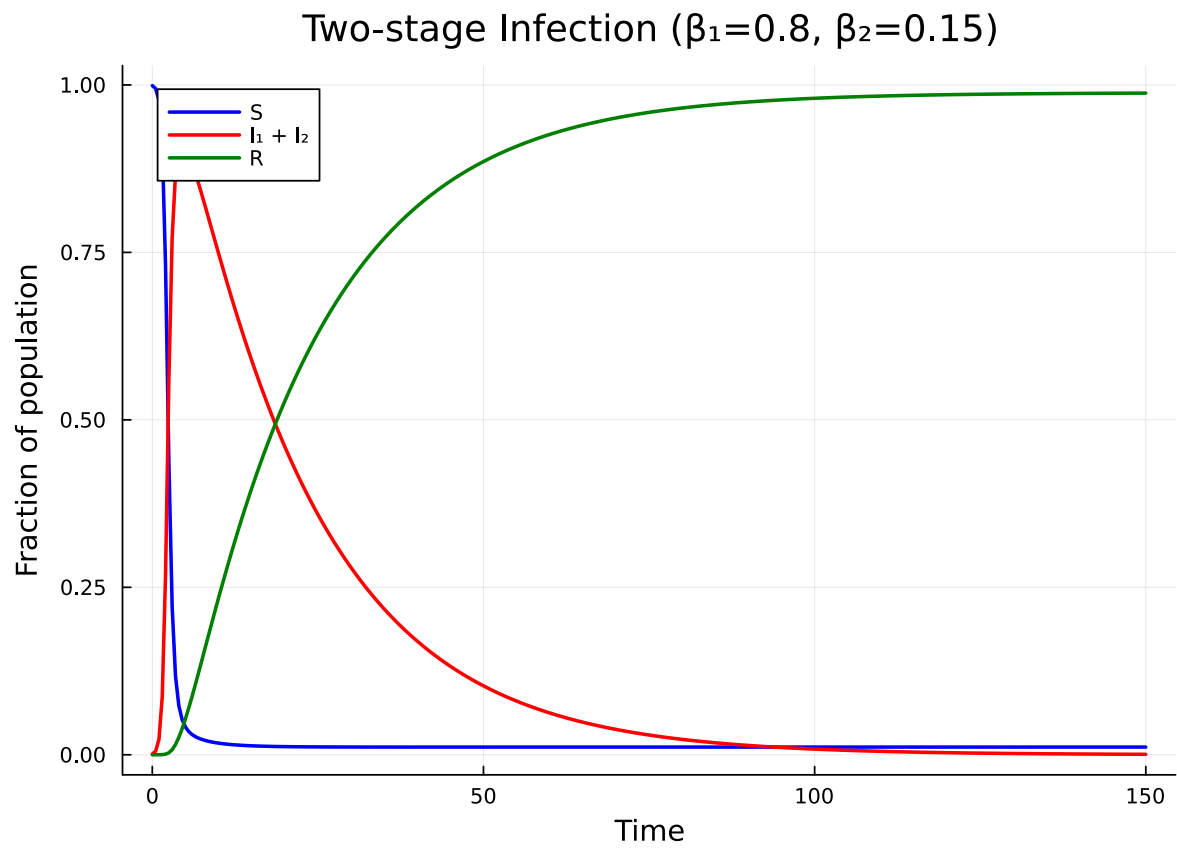


Figure 3: Figure 3: Two-stage infection with acute (high β) and chronic (low β) phases.

```

p = plot(xlabel = "Time", ylabel = "Edge probability",
         title = "Edge-level states ( $\phi$  variables)")
for (name, var) in sort(collect( $\phi$ _vars); by = first)
    short = replace(name, r"\(t\) "  $\Rightarrow$  "")
    plot!(p, sol_multi.t, sol_multi[var], label = short, linewidth = 2)
end
p

```

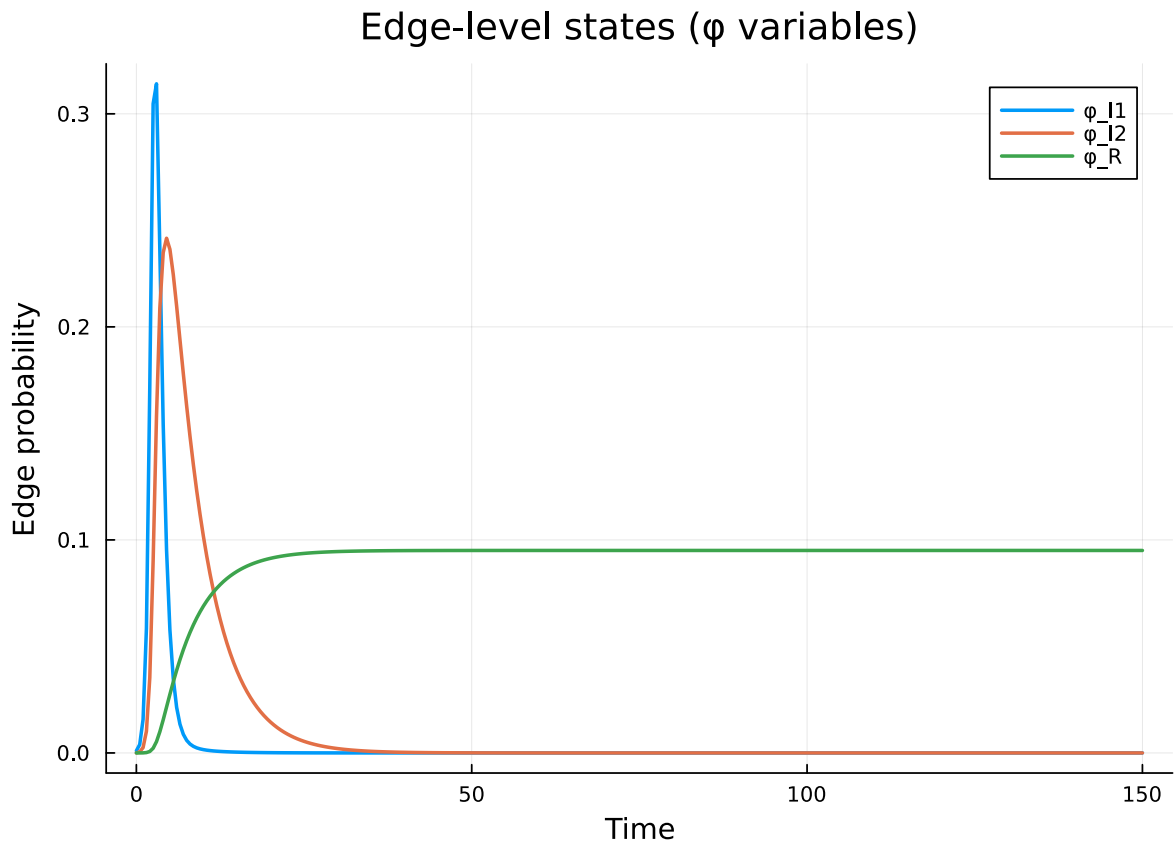


Figure 4: Figure 4: Edge-level probabilities for the two-stage infection model.

Simulation validation

We compare each ODE prediction (SIR, SEIR, SIS) against Gillespie SSA on a Poisson network with the same mean degree $\kappa = 5$ via `NetworkOutbreaks.jl`.

```

include("../_validation.jl")

builder = poisson_graph_builder(1000, κ_val)

# SIR ribbon (epidemic,  $t \in [0, 40]$ )
t_sir, μ_sir, σ_sir = gillespie_ribbon(
    sir_model(), Dict{:β ⇒ β_val, :γ ⇒ γ_val}, builder;
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = (0.0, 40.0), seed_fraction = seed_fraction)

# SEIR ribbon
t_seir, μ_seir, σ_seir = gillespie_ribbon(
    seir_model(), Dict{:β ⇒ β_val, :γ ⇒ γ_val, :σ ⇒ σ_val}, builder;
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = (0.0, 40.0), seed_fraction = seed_fraction, infected = :E)

# SIS ribbon (endemic, longer horizon)
t_sis_g, μ_sis_g, σ_sis_g = gillespie_ribbon(
    sis_model(), Dict{:β ⇒ β_val, :γ ⇒ γ_val}, builder;
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = (0.0, 80.0), seed_fraction = seed_fraction)

```

([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5 ... 75.5, 76.0, 76.5, 77.0, 77.5, 78.0, 78.5,

```

EI_ssa = μ_seir[:E] .+ μ_seir[:I]
σEI_ssa = sqrt.(σ_seir[:E].^2 .+ σ_seir[:I].^2)
plot(t_sir, μ_sir[:I], ribbon = σ_sir[:I],
     label = "SSA SIR I (mean ± 1σ)", color = :gray, fillalpha = 0.3)
plot!(sol_sir.t, I_sir, label = "EBCM SIR I", color = :red, linewidth = 2)
plot!(t_seir, EI_ssa, ribbon = σEI_ssa,
     label = "SSA SEIR E+I (mean ± 1σ)", color = :lightgray, fillalpha = 0.3)
plot!(sol_seir.t, EI_seir, label = "EBCM SEIR E+I", color = :orange, linewidth = 2)
xlabel!("Time"); ylabel!("Fraction infected")
title!("EBCM vs SSA on Poisson κ=5")

```

```

plot(t_sis_g, μ_sis_g[:I], ribbon = σ_sis_g[:I],
     label = "SSA SIS (mean ± 1σ)", color = :gray, fillalpha = 0.3)
plot!(sol_sis.t, I_sis, label = "EBCM SIS", color = :red, linewidth = 2)
xlabel!("Time"); ylabel!("Fraction infected")
title!("SIS endemic equilibrium")

```

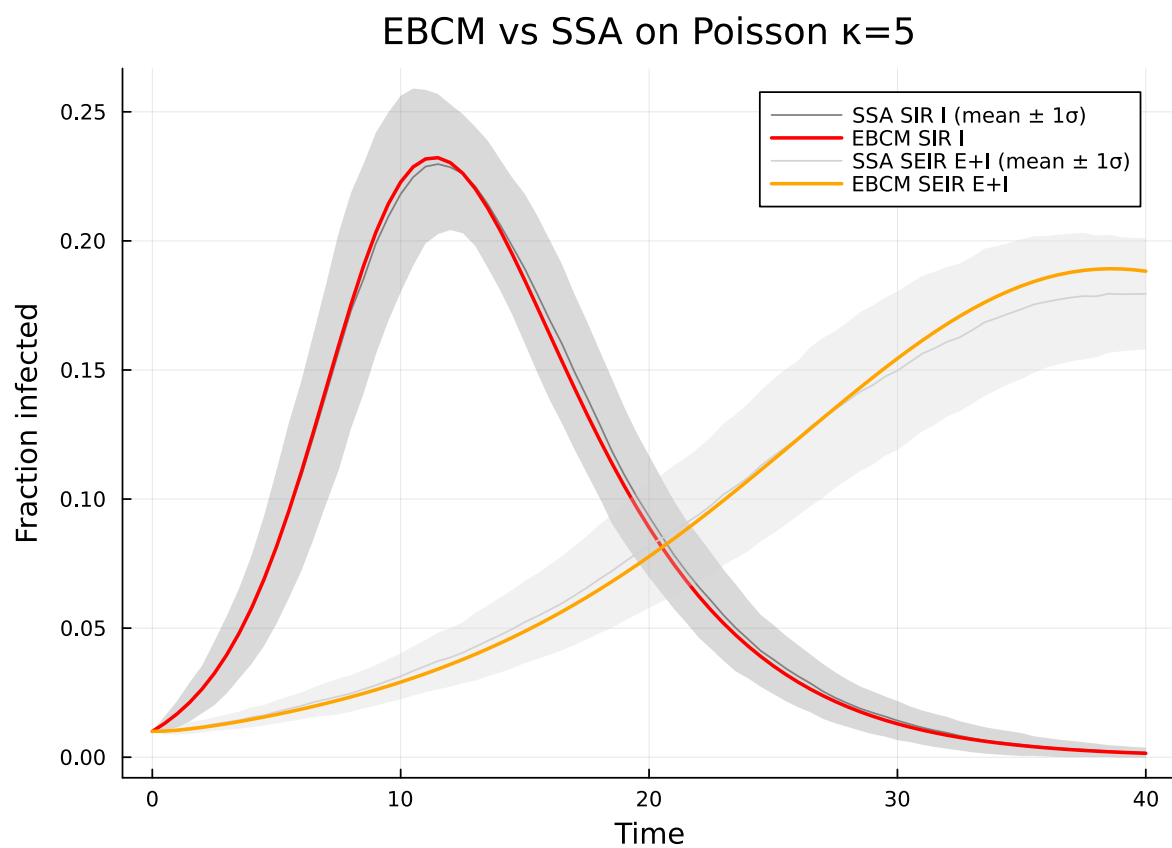


Figure 5: Figure 5: Gillespie SSA mean $\pm 1\sigma$ ribbon (gray) versus EBCM I/(E+I) curve (line) for SIR and SEIR.

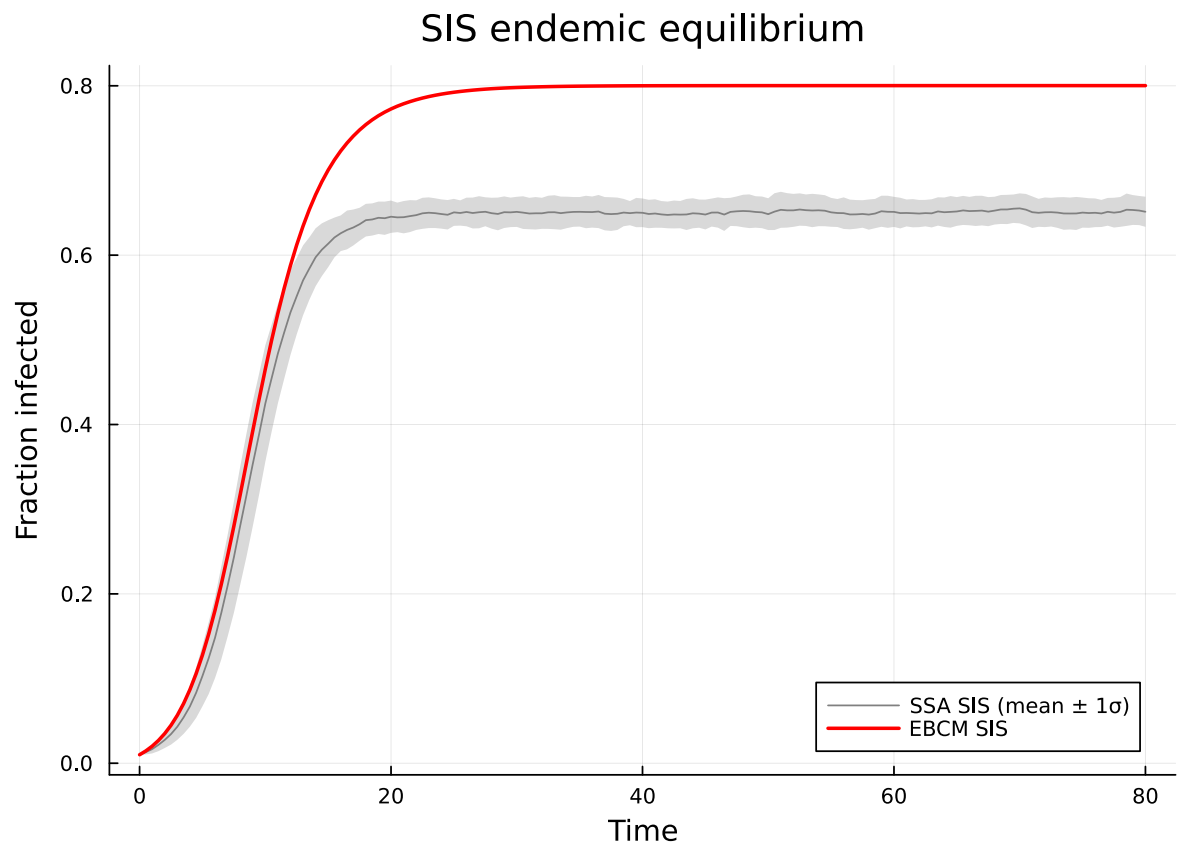


Figure 6: Figure 6: SIS on Poisson $\kappa=5$: EBCM endemic equilibrium versus SSA mean $\pm 1\sigma$.

The deterministic EBCM curves track the SSA ensemble means well across SIR, SEIR, and SIS, confirming that the configuration-model closure recovers the correct large-network limit.

Summary

- SIS: 1 ODE — produces endemic equilibria.
- Multi-stage: arbitrary disease graphs via `DiseaseProgression` with per-stage transmission rates.
- The package automates all the bookkeeping for edge-level variables, regardless of disease complexity.